Quine–McCluskey Minimisation

Kartike Sharan (3008620s)

1 Introduction

Boolean logic minimisation is an important task in digital system design, allowing complex logical expressions to be simplified in order to reduce hardware complexity and cost. While Karnaugh maps are commonly used for this purpose, they become impractical as the number of variables increases, motivating the need for a more systematic algorithmic approach.
In this project, a Boolean function of n variables is specified via user-entered truth table outputs, where each of the 2^n input combinations is labelled as 0, 1, or X (don't-care). The Quine–McCluskey algorithm is implemented in Python to automate the generation of a logically equivalent, simplified sum-of-products expression.

2 Algorithmic Approach

2.1 Input Specification and Truth Table Construction

The algorithm begins by requesting the number of Boolean variables, n, which determines the size of the problem. A complete truth table containing 2^n input combinations is constructed automatically. For each row, the user specifies an output value of 0, 1, or X.
Rows labelled 1 are treated as minterms, while rows labelled X are treated as don't-care terms. These values are extracted and converted into an internal representation for subsequent minimisation.

2.2 Implicant Representation

Each Boolean implicant is represented internally by a bit pattern and an associated mask indicating don't-care positions introduced during combination. The bit pattern encodes fixed variable values, while the mask allows insignificant variables to be ignored during comparison.
Each implicant also records the set of minterms it covers, enabling reliable identification of essential prime implicants during later stages of the algorithm.

2.3 Grouping and Combination of Terms

Implicants are grouped according to the number of logic 1s in their bit patterns, as only terms in adjacent groups can differ by a single variable. This restriction significantly reduces unnecessary comparisons while preserving correctness. Pairs of implicants that differ in exactly one unmasked variable are combined, introducing an additional don't-care position.
This process is applied iteratively until no further combinations are possible, ensuring that all valid higher-order implicants are generated.

2.4 Prime Implicant Identification and Cover Selection

Implicants that can no longer be combined during the grouping process are identified as prime implicants. Essential prime implicants, which are the only terms covering specific minterms, are selected first to guarantee correctness of the final expression. Any remaining uncovered minterms are then covered using a greedy selection strategy, in which prime implicants that cover the largest number of uncovered minterms are chosen.

2.5 Boolean Expression Generation

Once the final set of prime implicants has been selected, each implicant is converted into its corresponding symbolic Boolean term, and the simplified sum-of-products expression is produced.

3 Software Design and Implementation

The program is structured to separate user interaction, minimisation logic, and output generation into distinct stages, improving clarity and ease of modification. Input handling guides the user through truth table specification while enforcing valid entries and providing warnings for large problem sizes.

Appropriate data structures are used throughout the implementation. Sets are employed to prevent duplicate implicants during combination, while dictionaries are used to organise implicants by grouping criteria and coverage relationships. This design supports both clarity and robustness as problem size increases.

4 Results

The implementation was tested using a four-variable Boolean function defined by a combination of minterms and don't-care conditions. The algorithm produced the simplified expression

$$F = C'B + DC' + DB.$$

To verify correctness, the same function was minimised manually using a Karnaugh map. A comparison of the program output, Karnaugh map inputs, and the corresponding minimised expression is shown in Figure 1. The two results are algebraically equivalent, confirming the correctness of the implementation.

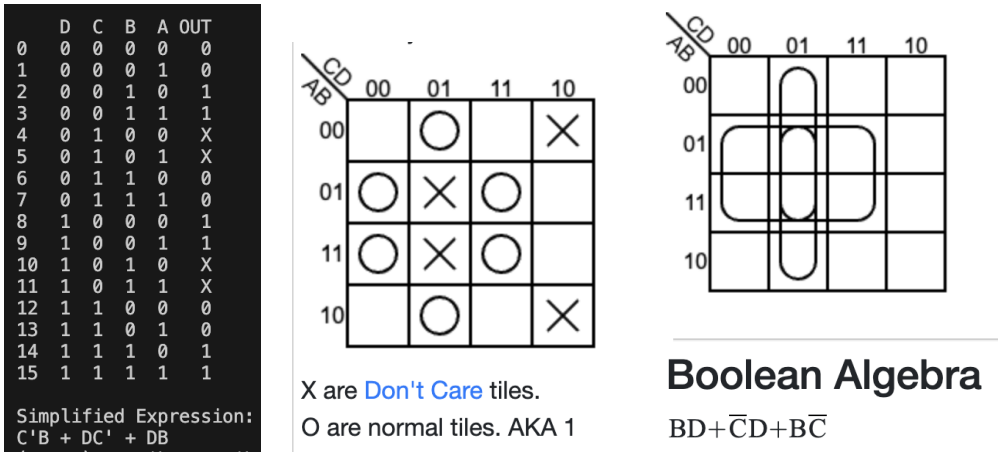


Figure 1: Comparison of the program-generated result (left), Karnaugh map input configuration with don't-care terms (centre), and the equivalent Karnaugh map minimised expression (right).

5 Evaluation and Limitations

The implementation performs reliably for small Boolean functions, but relies on a greedy prime implicant selection strategy that does not guarantee a globally minimal solution for larger problems (≥ 5 variables). As the number of variables increases, this can result in expressions containing more terms than strictly necessary.

Although the Quine–McCluskey algorithm is theoretically applicable to any number of variables, its exponential complexity causes computation time and memory usage to grow rapidly, making the approach impractical for larger inputs.

The program prioritises guided truth table entry with validation over a compact input format, reducing user error at the expense of scalability. This design choice was considered appropriate given the intended problem size.

6 Conclusion

This project demonstrated the successful implementation of the Quine–McCluskey algorithm to automate Boolean logic minimisation, producing results consistent with manual Karnaugh map solutions for small problem sizes. The program provides a clear and systematic approach to minimisation while supporting don't-care conditions and user-guided input.

Future improvements would focus on scalability and optimality, particularly through replacing the greedy selection strategy with Petrick's method to guarantee a minimal solution for larger problems. Additional enhancements could include alternative input modes and detection of exclusive-OR structures to allow further simplification.